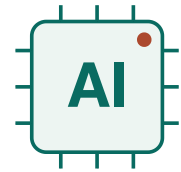


THIRD-YEAR ENGINEERING STUDENTS / ONE MONTH

Applied AI: From Data to RAG



APPLIED AI COURSE

A practical introduction to PyTorch, dataset validation, Transformers, Hugging Face and Retrieval-Augmented Generation.

4

Teaching weeks

8

Guided sessions

16

Contact hours

Monday: 2 hours offline at college**Friday or Saturday:** 2 hours on Google Meet each week

Course Purpose

Help students move from understanding data and neural networks to building a small, working AI application. Every week combines essential theory, guided coding and progress towards a team project.

Scope of the Course

- **PyTorch foundations:** tensors, automatic differentiation, datasets, data loaders and a basic neural-network training loop.
- **Dataset validation:** schema checks, missing values, duplicates, class balance, leakage prevention and trustworthy evaluation splits.
- **Transformers and Hugging Face:** tokenization, attention concepts, pretrained model inference and model/dataset cards.
- **RAG:** document preparation, embeddings, retrieval, grounded answers, source citations and basic evaluation.

Expected Level

Introductory and application-oriented. Suitable for students across engineering departments who have basic Python knowledge.

Outside This Month

Training large language models from scratch, advanced fine-tuning, distributed training and production-scale deployment.

Planning assumption: four teaching weeks, with one online session per week. Exact dates, timings and the Friday/Saturday choice will be confirmed with the college.

How the Month Works

A progressive sequence: validate data, train a small model, use pretrained models, then build a grounded assistant.

Week	Monday / Offline / 2 hours	Friday or Saturday / Meet / 2 hours
01	Dataset quality and validation	PyTorch tensors and training basics
02	Train and evaluate a small classifier	Transformers and Hugging Face
03	RAG architecture and document indexing	Retrieval, answer generation and citations
04	Project clinic, evaluation and improvements	Final demonstrations and assessment

Before the First Session

- Python: variables, loops, functions, lists and dictionaries.
- Basic familiarity with tables, averages, vectors and matrices.
- A short Python readiness exercise; preparatory material for students needing a refresher.
- No prior deep-learning experience required.

Student Setup

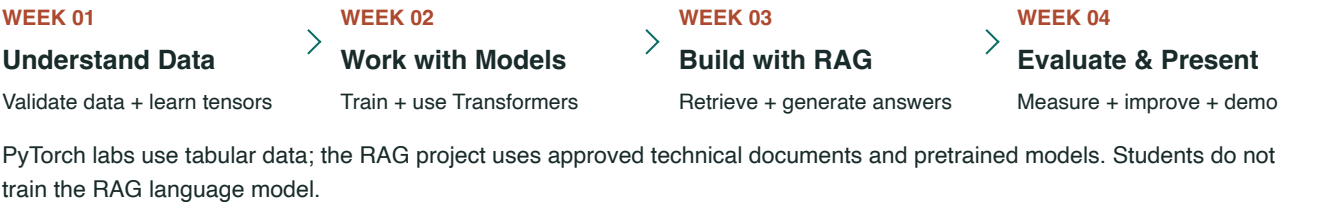
- Laptop, browser, internet and a Google account for Meet and Colab.
- Google Colab as the common notebook environment; local Jupyter is optional.
- Hugging Face account if required for selected resources.
- No dedicated GPU purchase required for the planned introductory labs.

Teaching and Practice Model

Offline: concepts, live demonstrations, paired coding and in-person troubleshooting. **Online:** short recaps, live notebook exercises, screen-sharing support and project reviews.

Between sessions: approximately 2-3 hours of student practice each week, additional to the 16 contact hours. Total expected effort is approximately **24-28 hours**.

The Learning Journey



Trainer and College Preparation

Provide tested starter notebooks, datasets, sample documents and evaluation questions before class. Arrange a projector, power outlets, dependable connectivity and the Meet link. Pre-download small model assets where feasible and prepare saved example outputs for connectivity interruptions.

Compute and cost: use small, publicly accessible models and datasets. Free cloud GPU availability is not guaranteed. Any paid API use requires prior approval; it is not a student prerequisite.

Foundations to Pretrained Models

Four sessions that establish reliable data handling and the core model workflow.

WEEK 01 / DATA QUALITY & PYTORCH

01 / Validate Before You Train

Monday / Offline

Theory: AI/ML workflow; features and labels; missing values, types, ranges, duplicates, imbalance and train/validation/test separation.

Practical: inspect a small dataset with pandas; add validation assertions; remove leakage risks; split data before fitting preprocessing.

Output: a dataset-quality checklist, validation notebook and reproducible split. Keep the test set untouched during model selection.

10 min readiness + 30 min theory + 60 min lab + 20 min review = 120 min

02 / Understand PyTorch Building Blocks

Friday/Saturday / Meet

Theory: tensors, shapes, devices, gradients, loss functions and optimizers; how a training step changes model parameters.

Practical: create tensors, inspect gradients and use Dataset/DataLoader; run a tiny neural network with a supplied starter notebook.

Output: a working forward/backward pass and a short explanation of the training loop. Practice: complete and rerun the notebook.

10 min recap + 30 min theory + 65 min lab + 15 min checkpoint = 120 min

WEEK 02 / EVALUATION & TRANSFORMERS

03 / Train, Measure and Improve

Monday / Offline

Theory: epochs, batches, overfitting, evaluation mode, accuracy, precision, recall and F1; why accuracy alone can mislead.

Practical: train a small classifier on validated data; compare with a simple baseline; inspect loss and a confusion matrix; save model weights.

Output: baseline comparison, validation metrics and error examples. Choose the model using validation data, then report the held-out test result.

15 min review + 25 min theory + 65 min lab + 15 min discussion = 120 min

04 / Use Transformers with Hugging Face

Friday/Saturday / Meet

Theory: tokens, embeddings, attention intuition, context limits and encoder/decoder roles; pretrained inference versus fine-tuning.

Practical: inspect a model card and license; load a tokenizer and small pretrained model through Transformers; run a text task and inspect failures.

Output: an inference notebook, model-choice note and capstone team/topic. Practice: collect approved project documents and draft questions.

10 min recap + 35 min theory + 60 min lab + 15 min project brief = 120 min

Build, Evaluate and Demonstrate

Apply pretrained models to a small, traceable document-question-answering system.

WEEK 03 / RETRIEVAL-AUGMENTED GENERATION

05 / Prepare Documents and Build Retrieval

Monday / Offline

Theory: why RAG is useful; documents, chunks, embeddings, similarity and top-k retrieval; RAG versus model training.

Practical: extract text from approved text-based documents; retain source/page metadata; split into chunks; embed and index them.

Output: a searchable document collection with source-linked results. Begin with a small in-memory index; FAISS is an optional extension.

10 min recap + 30 min theory + 65 min lab + 15 min checkpoint = 120 min

06 / Generate Grounded Answers

Friday/Saturday / Meet

Theory: retrieved context, prompt structure, grounded answers, hallucinations and the difference between a citation and supporting evidence.

Practical: connect retrieval to a small instruction model; show source references; test missing-information questions and an explicit abstention response.

Output: end-to-end RAG notebook. Practice: test ten questions and log incorrect retrieval, unsupported claims and weak citations separately.

10 min recap + 25 min theory + 65 min lab + 20 min project review = 120 min

WEEK 04 / PROJECT QUALITY & PRESENTATION

07 / Project Clinic and Evaluation

Monday / Offline

Theory: retrieval hit rate, answer correctness, evidence support, citation quality and response time; document privacy and prompt-injection risks.

Practical: run the fixed evaluation set; compare one chunk-size or top-k change; inspect failures; verify citations and missing-answer handling.

Output: a before/after evaluation table, corrected issues and a rehearsed demo. Keep a small question subset unseen until final assessment.

15 min review + 20 min evaluation briefing + 70 min clinic + 15 min rehearsal = 120 min

08 / Final Project Showcase

Friday/Saturday / Meet

Demonstration: explain the problem, data checks and RAG flow; show an answer with evidence, an unsupported question and one failure case.

Assessment: team demonstration, individual viva, evaluation results and feedback. Students explain limitations and their own contribution.

Output: final notebook, report, slides and completed assessment rubric.

10 min setup + 80 min demos/viva + 20 min feedback + 10 min next steps = 120 min

Showcase capacity: the plan accommodates up to eight teams at ten minutes each, including viva. For a larger cohort, arrange parallel assessors or an additional assessment slot before the course begins.

A Technical Knowledge Assistant

Build a small RAG assistant that answers questions from a verified collection of engineering documents and identifies its sources.

Recommended Project Brief

Teams of 3-4 students use approved lab manuals, equipment guides or technical notes: 5-10 short documents or about 20-50 text-based pages. Scanned-document OCR is outside the core project.

How the RAG Assistant Works

PREPARE ONCE / DOCUMENT INDEXING



AT QUESTION TIME / RETRIEVAL + GENERATION



Use the same embedding model for documents and questions. Evaluate retrieval, evidence support and citations.

Required Features

- **Data validation:** detect empty or duplicate text, confirm readable extraction and record document permissions and source information.
- **Traceable retrieval:** display the retrieved passages and retain document/page or section references.
- **Grounded responses:** provide an answer with supporting citations and say when the supplied documents do not contain enough information.
- **Evaluation:** use ten documented questions: six factual, two requiring multiple passages and two unanswerable from the collection.
- **Reproducibility:** record model IDs, package versions and settings; provide clear notebook execution steps. A web interface is optional.

Milestone	Team Submission
End of Week 2	Problem statement, approved documents, team roles and draft questions.
End of Week 3	Working RAG notebook, ten-question set and initial failure log.
End of Week 4	Final notebook, source manifest, evaluation table, 2-3 page report and 5-slide demonstration.

Responsible Use and Boundaries

Use public or college-approved content only. Do not upload student records, personal data, confidential documents or API keys. Check model and dataset licenses. Treat document instructions as untrusted input and test for prompt injection; prompting alone is not a security guarantee. The assistant is a learning prototype, not an authority for safety-critical engineering decisions.

Team accountability: divide data preparation, retrieval, answer generation and evaluation tasks, but require each member to explain the full pipeline.

What Students Take Away

Observable Learning Outcomes

1. Validate a dataset and explain how leakage compromises evaluation.
2. Build and evaluate a small neural network using PyTorch.
3. Explain tokenization and attention at an introductory level; run a pretrained Hugging Face model.
4. Implement a document-to-answer RAG workflow with traceable sources.
5. Evaluate retrieval and answers, report failures and demonstrate responsible data use.

Proposed Assessment / 100 Marks

Component	Marks	Evidence
Individual practical work	30	Data validation (10), PyTorch (10), Hugging Face inference (10).
Individual concept checks / viva	20	Data splits, training loop, Transformers, RAG and limitations.
Team capstone	40	Data quality (10), working pipeline (15), evaluation and grounding (15).
Demonstration and documentation	10	Clarity, reproducibility and explanation of contributions.

Suggested completion standard: attend at least 6 of 8 sessions, submit the required individual labs and project, and score at least 60/100. Attendance, certification and grading policy remain subject to college approval.

Simple RAG Evaluation Scorecard

For the eight answerable questions, report retrieval hit rate at a fixed top-k: questions with a relevant passage retrieved / 8. Separately review answer correctness, whether each material claim is supported, and whether citations point to the correct evidence. For the two unanswerable questions, record correct abstentions / 2. Report response time and settings. A ten-question test is instructional evidence, not a production-quality benchmark.

Academic and Career Scope

The course provides a foundation for department mini-projects, final-year project exploration and further AI/ML learning. Students finish with practical notebooks and a demonstrable prototype. One month does not establish professional AI-engineering proficiency or guarantee internship or placement outcomes.

Resource Materials & Course-Tracking Console

Resource materials will be provided as part of the course.

A dedicated console will be provided for students and faculty to track course progress.

After the course: deepen Python and statistics, expand the evaluation set, study fine-tuning fundamentals and improve the same project before attempting deployment.